

QuantBvh query_self_aabb_kernel 性能退化分析

现象

金字塔场景 (25251 个刚体) · query_self_aabb_kernel 耗时 :

阶段	耗时
金字塔稳定 (空中)	0.203 ms
落地过渡期	1.6 ms
完全静止在地面	3.2–3.9 ms

从稳定到静止 · 性能退化约 16 倍。

实验方法

在 query_self_aabb_kernel 旁添加无共享内存缓冲区的诊断 kernel · 统计每个线程的 :

- 遍历节点数 (nodes_visited)
- AABB 重叠测试通过数 (overlap_hits · 含内部节点 + 叶节点)
- 叶节点命中数 (leaf_hits)
- 实际输出碰撞对数 (pairs)

通过 atomicAdd/Max/Min 汇总全局统计量 · 每 300 帧采样一次。

实验数据

指标	空中 (第 1 次采样)	落地静止 (稳态平均)	变化倍数
kernel_time	0.203 ms	~3.3 ms	×16
avg_nodes/thread	66.0	~197	×3.0
max_nodes/thread	1,205	~22,500	×18.7
stddev_nodes	15.1	~160	×10.6
total_overlap_hits	995,484	~2,780,000	×2.8
avg_overlap_hits/thread	39.4	~110	×2.8
total_leaf_hits	174,751	~306,000	×1.8
total_pairs	74,750	~140,500	×1.9

根因分析

1. Warp Divergence 是主因 (×10+ 贡献)

这是最关键的发现。看两个数字 :

```
空中:  avg=66,  max=1205,  stddev=15.1    (max/avg = 18x)
落地:  avg=197, max=22500, stddev=160      (max/avg = 114x)
```

stackless BVH 遍历的工作方式是：**warp 内 32 个线程独立遍历树**，每个线程可能走不同长度的路径。但 CUDA warp 是 SIMT——warp 只在**最慢的线程完成后**才能释放。

- **空中**：stddev 仅 15.1， $\text{max/avg} = 18\times$ 。线程间工作量差异小，warp 内几乎同步完成。
- **落地**：stddev 达 160， $\text{max/avg} = 114\times$ 。少数线程（密集堆叠区域的 body）要遍历 22000+ 个节点，而同一 warp 中的其他线程可能只需 17 个节点。**这些快线程必须空等慢线程**，造成了大量的计算资源浪费。

定量估算：如果没有 divergence，仅按 `avg_nodes` 增长 ($\times 3$)，时间应为 $0.203 \times 3 \approx 0.6 \text{ ms}$ 。实际 **3.3 ms**，说明 warp divergence 贡献了约 $3.3 / 0.6 \approx 5.5\times$ 的额外开销。

2. AABB 重叠密度增大 ($\times 3$ 贡献)

```
空中:  avg 66 nodes/thread, avg 39.4 overlap_hits
落地:  avg 197 nodes/thread, avg 110 overlap_hits
```

当金字塔在空中时，body 沿 Z 轴（重力方向）分布范围大，AABB 在 quantized 空间中分散。BVH 内部节点的 AABB 粗粒度覆盖不同的物理区域，大量查询在高层就被 **early-reject**。

落地后，所有 body 被压缩到地面附近薄薄的一层。BVH 内部节点的 AABB 在 **Z 方向严重重叠**，quantized overlap test 无法有效剪枝，导致遍历必须深入更多的子树。

3. 空间聚类导致 BVH 树退化

Morton code 排序基于 3D 空间均匀量化。当 25000 个 body 全部堆叠在地面的薄层中：

- Z 方向的有效分辨率只占 14-bit 量化范围的很小比例
- Morton code 的 Z 分量几乎不提供区分度
- 大量 body 获得**相同或相邻的 Morton code**
- 结果：BVH 树在 Z 方向的空间分割质量极差，内部节点 AABB 大量重叠

这直接导致了每个 leaf 查询时要“穿过”更多无法被剪枝的内部节点。

4. 碰撞对数量几乎翻倍

```
空中:  74,750 pairs
落地:  ~140,500 pairs
```

这本身不是 kernel 慢的主要原因（pairs 增长 $\times 1.9$ ，远小于时间增长 $\times 16$ ），但更多的 pairs 意味着：

- 更多的叶节点 hit 处理
- 更多的 `atomicAdd` 到 shared buffer
- 更频繁的 shared buffer flush 到 global memory

各因素贡献总结

总退化：×16

|— 平均工作量增加 (avg_nodes ×3.0)

→ ×3.0

|— Warp divergence (max/avg 严重失衡)

→ ×5.0~5.5

| |— 少数线程遍历 22000+ 节点拖累整个 warp

|— 输出 pairs 增加 (×1.9)

→ ~×1.0 (minor)

|— 其他因素 (cache pressure, atomics)

→ ~×1.0 (minor)

根源

因素	空中	落地
body 空间分布	3D 均匀 (金字塔结构)	2D 扁平 (薄层堆叠)
Z 方向范围	大 (0 ~ 50m)	小 (0 ~ 2m)
Morton code 区分度	高 (3 轴都有效)	低 (Z 轴退化)
BVH 内部节点 AABB	紧凑，剪枝率高	重叠严重，剪枝率低
线程工作量分布	均匀 (stddev/avg=23%)	极不均匀 (stddev/avg=81%)
Warp 效率	高	低 (大量空闲等待)

可能的优化方向

1. **Warp-level 工作窃取/重分配**：让遍历完的线程帮助同 warp 内未完成的线程，减少尾延迟。
2. **Persistent thread + 任务队列**：取代 one-leaf-per-thread 模型，用全局任务队列动态分配叶查询。
3. **Z-curve → Hilbert curve**：Hilbert 曲线在扁平分布下可能提供更好的空间局部性。
4. **自适应量化**：检测空间分布退化时，仅对有效维度 (X-Y) 使用更多量化位，或分层 BVH。
5. **Top-level spatial hash + BVH 混合**：先用 uniform grid 粗分，每个 cell 内独立 BVH，避免全局树的退化。